
Basics of Internet Technologies 2

Back-end Programming and Database Management

Lecture 01

Introduction to C# and .NET

Learning Objectives

- Explain what programming is in practical terms.
- Describe what backend development is responsible for.
- Separate C# from .NET conceptually.
- Create and run console applications.
- Read data from the console and convert text into numbers.
- Use variables, constants, basic types, operators, simple conditions, and arrays.
- Understand how the semester project will grow during the course.

Lesson Timing Plan

- 10 min - programming, backend, C#, .NET, SDK, runtime.
- 15 min - variables, constants, naming, basic types.
- 20 min - console input, text values, conversion, Parse, TryParse.
- 15 min - arithmetic, precedence, formatting, common numeric mistakes.
- 15 min - simple conditions and boolean logic as a preview.
- 20 min - arrays: one-dimensional, numeric, rectangular, jagged.
- 10 min - semester project, milestones, exercises, questions.

Why This Matters

- A backend receives input, makes decisions, stores data, and returns useful output.
- Before APIs, databases, and authentication, students need confidence with ordinary code.
- Console applications are simple, visible, and perfect for learning the core ideas without web complexity.
- Every later backend concept reuses the same thinking: input -> processing -> output.

What Is Programming?

- Programming means giving precise instructions to a computer.
- A program reads data, stores values, performs operations, chooses paths, and produces results.
- Computers do exactly what the code says, not what the programmer hoped it meant.
- Good beginner code should be clear before it is clever.

What Is Backend Development?

- Backend code usually runs on a server or behind an application interface.
- It handles rules, data, users, security, persistence, and integration with other systems.
- A backend often exposes an API that clients can call over HTTP.
- In this course we start locally, then gradually move toward real web backends.

C# vs .NET

C#

- A programming language.
- Defines syntax and language features.
- Examples: variables, types, if, loops, arrays, classes, methods.

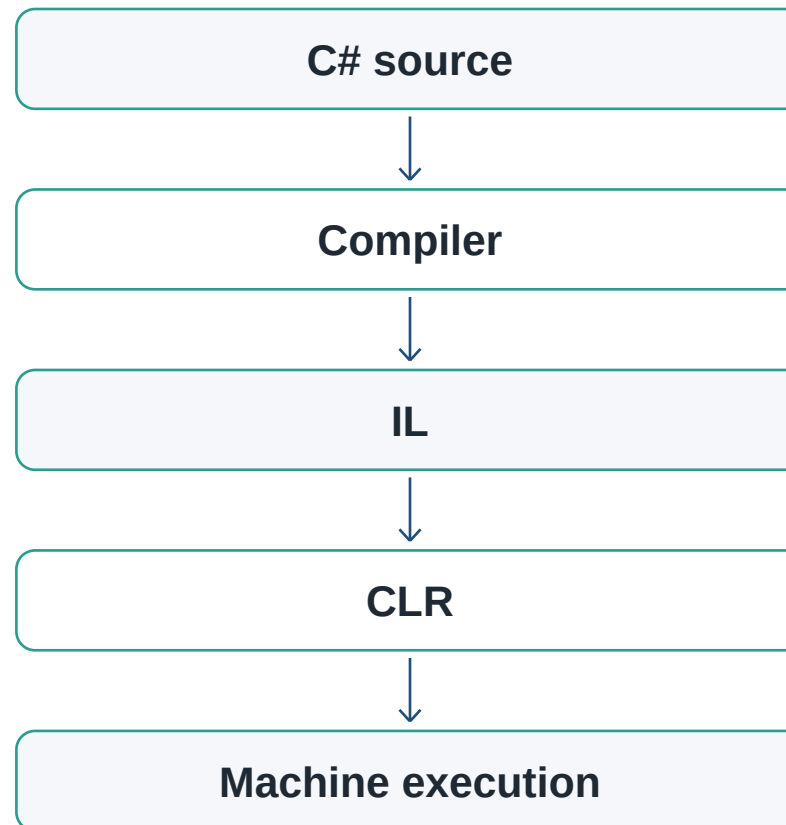
.NET

- A development platform and runtime ecosystem.
- Provides libraries, tools, compilers, and runtime support.
- Used for console apps, web APIs, desktop apps, games, cloud services, and more.

.NET SDK, Runtime, and CLR

- The SDK is used to create, build, run, and publish applications.
- The runtime is used to execute compiled .NET applications.
- The CLR is the Common Language Runtime. It manages execution of .NET code.
- For this course the selected SDK is .NET 10.0.302.

From Source Code to Execution



We only need the big picture today. JIT internals can wait.

Console Applications

- A console application runs in a terminal window.
- It is useful for learning because input and output are easy to see.
- The first file beginners usually meet is Program.cs.
- Top-level statements let us write the first program without defining a class manually.

Live Demo: Personal Expense Entry

```
Expense name: Coffee
```

```
Amount: 4.50
```

```
Currency: EUR
```

```
You spent 4.50 EUR on Coffee.
```

- This demo is intentionally tiny.
- It shows input, variables, parsing, and formatted output.
- Later lectures will turn the same idea into a menu, a collection, a stored file, a database, and an API.

Runnable Code Walkthrough Project

- Lecture 01 also includes a separate runnable project: `Lecture01.CodeBasics`.
- This project is not a bigger application. It is a guided code laboratory.
- It prints sections for variables, types, constants, operators, input, conversion, conditions, and arrays.
- The teacher can run it slowly and connect each output line to the source code.

Variables: Names for Values

```
string expenseName = "Coffee";  
int quantity = 2;  
decimal price = 4.50m;  
bool isNecessary = false;  
char currencyLetter = 'E';
```

- A variable is a named place for a value.
- The type controls what kind of value can be stored.
- The name should explain the meaning of the value in the program.

Common Beginner Types

```
byte smallWholeNumber = 25;  
int wholeNumber = 1000;  
long largeWholeNumber = 50000000000L;  
float approximateFloat = 3.14f;  
double approximateDouble = 3.1415926535;  
decimal moneyValue = 19.99m;
```

- Whole-number types store values without a fractional part.
- float and double are approximate floating-point types.
- decimal is usually a better beginner choice for money-like examples.

Text, Characters, and Boolean Values

```
string title = "Coffee";  
char firstLetter = 'C';  
bool isPaid = false;  
bool isExpensive = price > 100;
```

- string stores text with zero, one, or many characters.
- char stores one character.
- bool stores true or false and is used by conditions.

Constants

```
const decimal VatRate = 0.25m;  
const string DefaultCurrency = "EUR";  
  
decimal vat = subtotal * VatRate;
```

- A constant is a value that should not change while the program runs.
- Constants make important fixed values visible and named.
- Use constants for values such as default currency, tax rates, and fixed limits.

Naming Rules and Naming Style

- Variable names should describe meaning: price, quantity, productName.
- Avoid names that only describe type: text, number, data.
- Local variables usually use camelCase.
- Constants often use PascalCase in simple C# teaching examples.
- Names cannot contain spaces and cannot start with a digit.

Arithmetic Operators

```
decimal subtotal = quantity * price;  
decimal vat = subtotal * VatRate;  
decimal total = subtotal + vat;  
  
Console.WriteLine(total);
```

- Operators create new values from existing values.
- Common arithmetic operators are +, -, *, /, and %.
- The result type matters: int division and decimal division behave differently.

Integer Division vs Decimal Division

```
Console.WriteLine(10 / 3);  
Console.WriteLine(10m / 3m);
```

- $10 / 3$ uses int values, so the result is 3.
- $10m / 3m$ uses decimal values, so the result can include a fractional part.
- This is a common beginner bug when calculating averages, prices, or percentages.

Operator Precedence

```
int firstResult = 2 + 3 * 4;  
int secondResult = (2 + 3) * 4;
```

- Multiplication happens before addition.
- Parentheses make the intended order obvious.
- Use parentheses when it helps a beginner reader understand the calculation.

Console Output

```
Console.Write("Product name: " );  
Console.WriteLine("Saved.");  
Console.WriteLine(total);
```

- Console.Write prints without moving to the next line.
- Console.WriteLine prints and then moves to the next line.
- Clear prompts help the user know what to enter.

Console Input Is Text

```
Console.Write("Enter price: " );  
string priceText = Console.ReadLine();  
  
Console.WriteLine(priceText);
```

- Console.ReadLine reads what the user typed.
- The returned value is text, even if the user typed digits.
- Text must be converted before it can be used as a number.

Parsing Text to Numbers

```
string priceText = Console.ReadLine();  
decimal price = decimal.Parse(priceText, CultureInfo.InvariantCulture);  
  
string quantityText = Console.ReadLine();  
int quantity = int.Parse(quantityText, CultureInfo.InvariantCulture);
```

- Parse converts text into a specific type.
- decimal.Parse converts text to decimal.
- int.Parse converts text to int.
- Invalid text causes an error, which is useful to demonstrate but not user-friendly yet.

Culture and Decimal Separators

```
decimal price = decimal.Parse(priceText, CultureInfo.InvariantCulture);  
Console.WriteLine(price.ToString("F2", CultureInfo.InvariantCulture));
```

- Some systems use comma as a decimal separator, others use dot.
- For teaching examples we use invariant culture so 4.50 stays 4.50.
- This makes demo output stable on different computers.

Safer Conversion with TryParse

```
bool ok = int.TryParse(  
    discountText,  
    CultureInfo.InvariantCulture,  
    out int discountPercent);  
  
if (ok)  
{  
    Console.WriteLine(discountPercent);  
}
```

- TryParse tries to convert text without crashing the program.
- It returns true when conversion succeeds and false when it fails.
- The converted value is placed into the out variable.

Simple Conditions

```
if (enteredPrice <= 0)
{
    Console.WriteLine("Price must be greater than zero.");
}
else
{
    Console.WriteLine("Price accepted.");
}
```

- A condition lets the program choose a path.
- The expression inside if must produce true or false.
- Lecture 02 will return to conditions and use them in menus.

Comparison Operators

- `==` means equal to.
- `!=` means not equal to.
- `>` and `<` compare greater than and less than.
- `>=` and `<=` include equality.
- A comparison produces a bool value: true or false.

Logical Operators

```
bool priceIsPositive = enteredPrice > 0;  
bool quantityIsPositive = enteredQuantity > 0;  
bool canSave = priceIsPositive && quantityIsPositive;  
bool needsAttention = enteredTotal > 100 || discountPercent > 30;
```

- && means both sides must be true.
- || means at least one side must be true.
- ! reverses a bool value.

String Concatenation

```
string sentence = "You entered " + quantity + " item(s) of " + productName + ".";
```

- Concatenation joins text pieces together with +.
- Numbers can be joined into text for output.
- Long concatenation becomes difficult to read.

Helpful Syntax: String Interpolation

```
string sentence = $"You entered {quantity} item(s) of {productName}.";
```

- String interpolation starts with \$ before the string.
- Values are placed inside braces.
- This is easier to read than long concatenation.

Helpful Syntax: var

```
var title = "Coffee";  
var count = 3;  
var total = 12.50m;
```

- var lets the compiler infer the type from the assigned value.
- The variable still has a real type.
- Use explicit types while learning, then introduce var when the type is obvious.

One-dimensional Arrays

```
string[] productNames = new string[3];  
productNames[0] = "Coffee";  
productNames[1] = "Tea";  
productNames[2] = productName;
```

- An array stores several values of the same type.
- Array indexes start at 0.
- The Length property tells how many elements the array has.

Array Indexing

```
Console.WriteLine(productNames[0]);  
Console.WriteLine(productNames[1]);  
Console.WriteLine(productNames[2]);  
Console.WriteLine(productNames.Length);
```

- `productNames[0]` reads the first element.
- `productNames[2]` reads the third element.
- Using an index outside the array causes an error.

Array Initializer Syntax

```
decimal[] prices = { 4.50m, 3.20m, enteredPrice };  
  
decimal priceSum = prices[0] + prices[1] + prices[2];
```

- Initializer syntax creates an array with known starting values.
- The array size is inferred from the number of values.
- This is useful for small examples and fixed test data.

for Loop Over an Array

```
for (int index = 0; index < productNames.Length; index++)  
{  
    Console.WriteLine(productNames[index]);  
}
```

- A for loop is useful when the index matters.
- The loop starts at index 0.
- The loop continues while index is less than array Length.

foreach Loop Over an Array

```
foreach (string name in productNames)
{
    Console.WriteLine(name);
}
```

- foreach reads each value in order.
- It is simpler when the index is not needed.
- Lecture 02 and Lecture 03 will practice loops and collections more deeply.

Two-dimensional Rectangular Arrays

```
int[,] weeklySales = new int[2, 3];  
weeklySales[0, 0] = 5;  
weeklySales[1, 2] = 6;  
  
Console.WriteLine(weeklySales.GetLength(0));  
Console.WriteLine(weeklySales.GetLength(1));
```

- A rectangular array has rows and columns.
- Use two indexes: row and column.
- GetLength(0) returns row count. GetLength(1) returns column count.

Jagged Arrays

```
string[][] projectIdeas = new string[2][];  
projectIdeas[0] = new string[] { "ATM", "Expense Tracker" };  
projectIdeas[1] = new string[] { "Vet Clinic", "Task Manager", "Library" };
```

- A jagged array is an array of arrays.
- Inner arrays can have different lengths.
- This is different from a rectangular array.

Common Mistakes Today

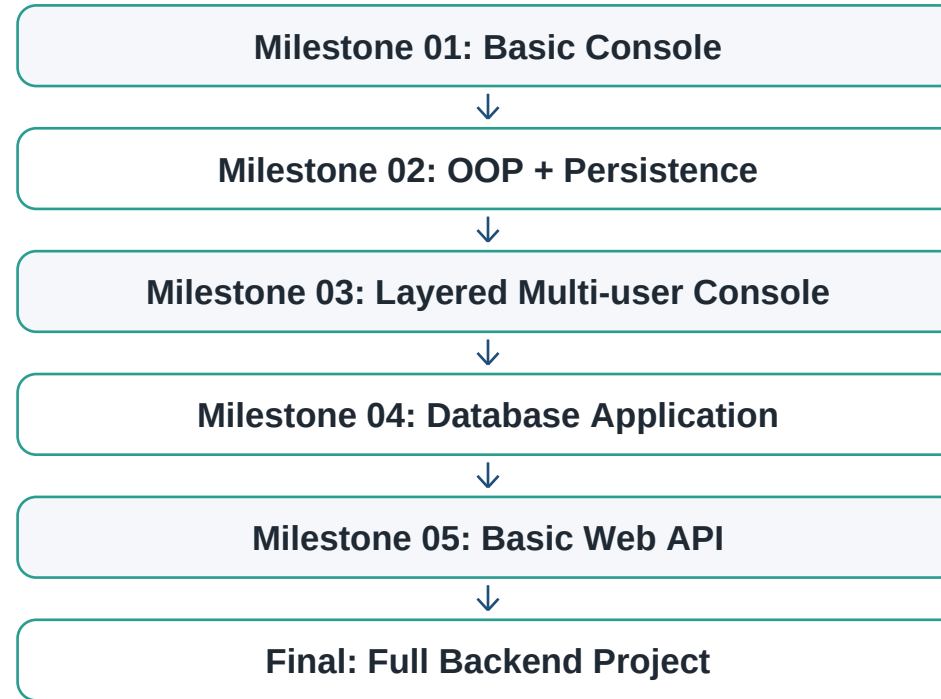
- Confusing C# the language with .NET the platform.
- Forgetting that `Console.ReadLine` returns text.
- Trying to calculate with text before conversion.
- Using comma instead of dot when the code expects invariant culture.
- Using array index 3 in an array that has indexes 0, 1, and 2.
- Trying to add architecture before the program has a real problem to solve.

The Semester Project

During this course you will build one application of your own.

- You will start with a very small console application.
- After almost every lecture you will improve it using the concepts learned during that lecture.
- During the final assessment you will show both the evolution of your application and its final backend version.
- The first version is single-user. Users, authentication, roles, and authorization come later.

Project Evolution Milestones



Students do not need twenty full copies. Keep important snapshot versions that can be shown during the defense.

Assessment Has Two Parts

Part A - Project Evolution

- Show milestone versions.
- Explain what changed.
- Explain what problem each new technique solved.

Part B - Final Project

- Run the final backend application.
- Show database, API, login, CRUD, validation, ownership, and code structure.
- Connect the final result to the semester evolution.

Possible Project Topics 1-14

- 1. ATM Simulator
- 2. Task Manager
- 3. Veterinary Clinic
- 4. Personal Library
- 5. Expense Tracker
- 6. Habit Tracker
- 7. Workout Tracker
- 8. Movie Collection
- 9. Recipe Manager
- 10. Study Planner
- 11. Inventory Manager
- 12. Event Planner
- 13. Hotel Booking System
- 14. Cinema Booking

Possible Project Topics 15-28

- 15. Restaurant Reservation System
- 16. Food Delivery Simulator
- 17. Car Rental
- 18. Parking Manager
- 19. Car Maintenance Tracker
- 20. Bicycle Service
- 21. Game Collection
- 22. Achievement Tracker
- 23. Music Collection
- 24. Playlist Manager
- 25. Notes Application
- 26. Password Vault Simulation
- 27. Contacts Manager
- 28. Calendar Application

Possible Project Topics 29-40

- 29. Course Registration System
- 30. Grade Tracker
- 31. Library Lending System
- 32. Warehouse Management
- 33. Online Shop
- 34. Support Ticket System
- 35. Bug Tracker
- 36. Project Management System
- 37. Pet Care Tracker
- 38. Medication Schedule
- 39. Travel Planner
- 40. Simple Social Posts Application
- Students may suggest their own topic after teacher approval.

Student Project Increment

- Choose a topic.
- Create a separate Visual Studio Solution for your own project.
- Give the application a clear name.
- Write 3-5 sentences about the purpose of the program.
- Define the first main data type.
- Create a Console project.
- Read 2-3 values through the console and convert numeric input.
- Optionally store several starting values in an array and print them.

Example First Data Type Ideas

Project

- Expense Tracker
- Task Manager
- Veterinary Clinic
- ATM Simulator

First Data

- Expense name + amount
- Task title + due date
- Pet name + visit reason
- Balance + deposit amount

Exercises and Questions

- Explain the difference between C# and .NET in your own words.
- Create a console application that reads a name, one decimal number, and one whole number.
- Convert the entered text into useful numeric types.
- Print calculations using concatenation and string interpolation.
- Create an array with at least three values from your project domain.
- Write a short paragraph describing your semester project idea.

Summary

- Programming is precise instruction writing.
- Backend development focuses on data, rules, users, persistence, and APIs.
- C# is the language; .NET is the platform and runtime ecosystem.
- Console input arrives as text and often needs conversion.
- Variables, operators, conditions, and arrays are the base for the next lectures.
- The semester project starts now and grows step by step into a real backend.